Autonomous Systems Report

Group Project: Sub-Terrain Challenge

Group 5: Chuan Yan, Wenxuan Fu, Aopu Wen, Zhuoya Zhang, Junda Zhu

1. Introduction

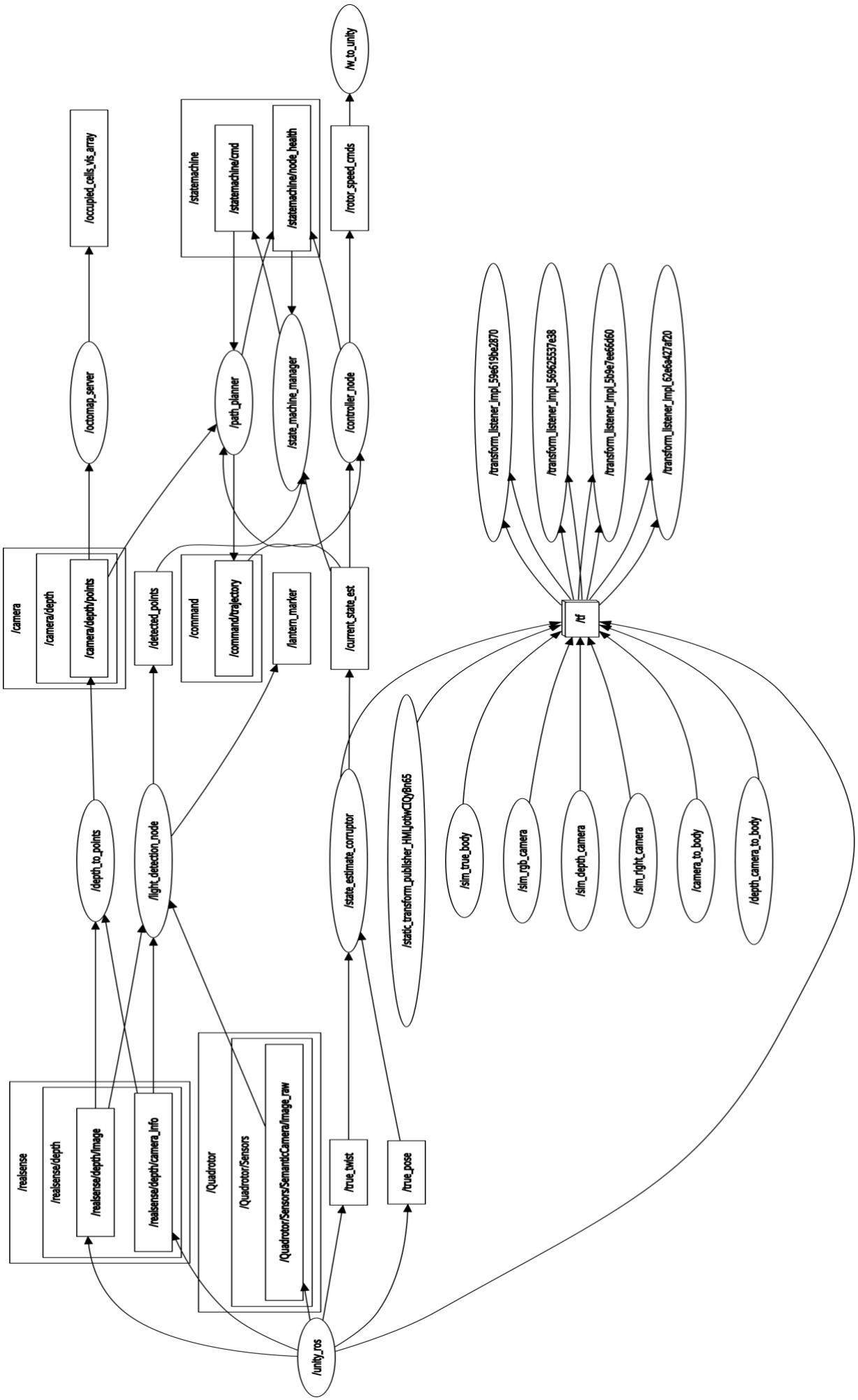
The project aims to locate four objects of interest in a cave environment as fast as possible and determine their positions. At the same time, a 3D representation of the cave needs to be generated. The project is based on quadrotor simulated in Unity. This documentation further explains the overall structure, implementation of main components and a bibliography.

2. Overall Structure

- **Simulation**: reads Sensor data from unity and sends commands to it. (already have)
- **Perception**: implement point cloud and OctoMap generation, implement target detection for lanterns, and return coordinates.(Session 4.1)
- **Exploration** :implements a frontier-based autonomous exploration strategy using OctoMap and density-based clustering to generate adaptive exploration goals in unknown 3D environments. (Session 4.2)
- **State machine:** orchestrates autonomous transitions between mission phases by processing real-time feedback. (Session 4.3)
- **Trajectory generation**: produces a smooth, collision-free flight path by calculating a series of time-indexed setpoints for the controller to follow.(already have)
- **Controller:** receives information of trajectory and calculates the rotor speeds of quadrotor to follow the desired trajectory. (Session 4.4)

3. Task allocation

- Chuan Yan: Octopmap geration and lantern detection
- Wenxuan Fu: generation of point cloud and Octomap, detection of target object
- Aopu Wen: cave exploration and navigation
- Zhuoya Zhang: state machine and trajectory generation
- Junda Zhu: controller and basic waypoint package


/occupied_cells_vis_array
/statemachine
/statemachine/cmd
/statemachine/node_health
/rotor_speed_cmds
/w_to_unity
/octomap_server
/path_planner
/state_machine_manager
/controller_node
/transform_listener_impl_59e619be2870
/transform_listener_impl_5696255337e38
/transform_listener_impl_5b9e7ee66d60
/transform_listener_impl_62e6a427af20
/camera
/camera/depth
/camera/depth/points
/detected_points
/command
/command/trajectory
/lantern_marker
/current_state_est
/tf
/depth_to_points
/flight_detection_node
/state_estimate_corruptor
/static_transform_publisher_HMLjotiwCIQyBn65
/sim_true_body
/sim_rgb_camera
/sim_depth_camera
/sim_right_camera
/camera_to_body
/depth_camera_to_body
/realsense
/realsense/depth
/realsense/depth/image
/realsense/depth/camera_info
/Quadrotor
/Quadrotor/Sensors
/Quadrotor/Sensors/SemanticCamera/image_raw
/true_twist
/true_pose
/unity_ros

4. Main Components

This chapter breaks down our system's main components, explaining our technical approach and highlighting the specific contributions of each team member.

4.1 Perception

The perception process starts from the depth camera mounted on the drone in the Unity simulation. The depth camera provides depth images that describe the distance between the drone and the surrounding environment.

4.1.1 Cloud pints and Octomap

To obtain 3D information from the 2D depth images, we implement a dedicated conversion node called /point_cloud_xyz_node. This node is responsible for converting depth images into a 3D point cloud. The main steps are as follows:

1. It subscribes to the depth image topic and the corresponding camera intrinsic parameters.
2. Each depth pixel is back-projected into 3D space using the camera projection model.
3. The reconstructed 3D points are packaged into a sensor_msgs/PointCloud2 message.
4. The point cloud is assigned the correct coordinate frame (e.g., Quadrotor/Sensors/DepthCamera).

Since the depth camera is attached to a moving drone, the generated point cloud is initially expressed in the local camera frame. However, for mapping and navigation, all spatial data must be represented in a common global coordinate system.

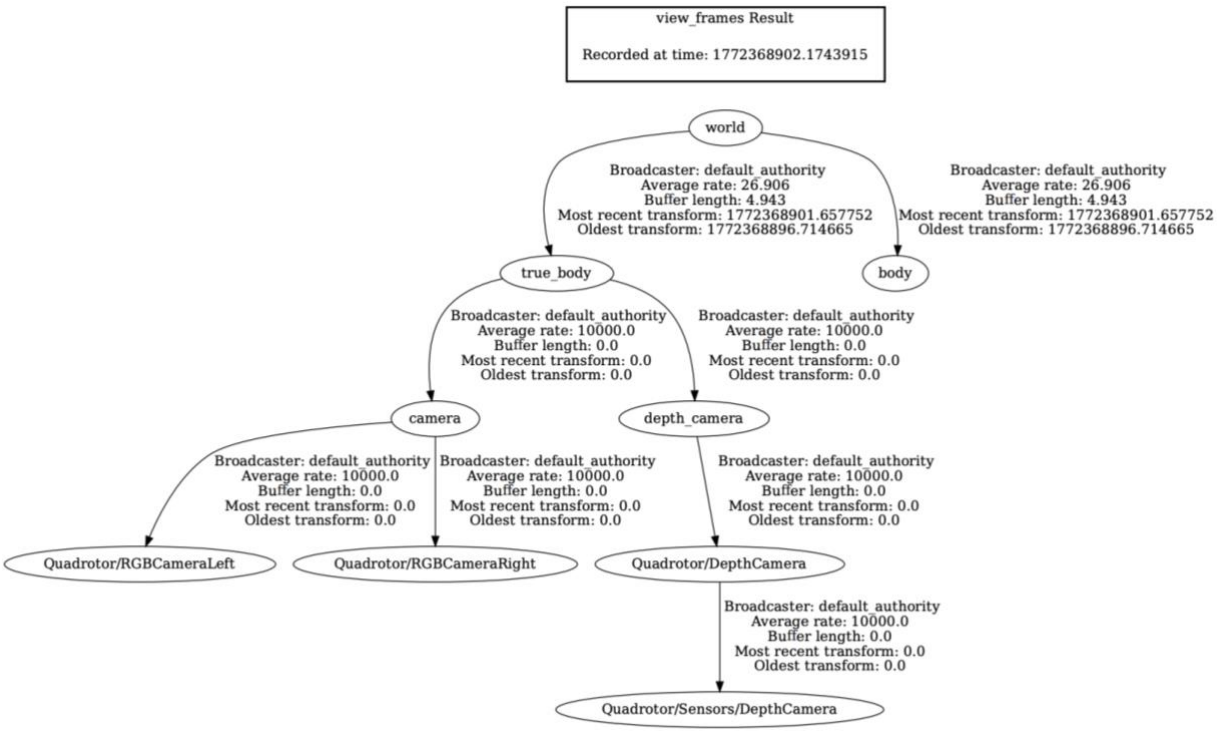


Figure 1: TF tree

The TF tree used in our system is shown in Figure 1. It illustrates the transformation chain from the global world frame to the drone body and further to each camera frame. This structure ensures that all sensor data can be consistently transformed into the global coordinate system for mapping and navigation.

The generated point cloud is provided to the octomap_server_node, which incrementally constructs a 3D occupancy map using an octree-based representation. Each incoming point cloud is first transformed from the camera frame to the global world frame via the TF tree. A ray-casting process updates the occupancy probability of voxels along each measurement ray, marking free space and occupied cells accordingly. The resulting OctoMap provides a probabilistic volumetric representation of the environment for obstacle avoidance and navigation.

4.1.2 Lantern detection

Lantern detection is performed by the light_detection_node, which processes camera images to identify characteristic bright regions corresponding to lanterns. After segmenting candidate regions based on color and intensity cues, the associated depth values are used to back-project the detected pixels into 3D coordinates in the camera frame. These positions are then transformed into the global coordinate frame and published as markers, enabling downstream navigation modules to use the lantern locations as semantic targets.

4.2. Exploration

This module implements an exploration strategy in ROS2 for autonomous navigation in unknown 3D environments using an OctoMap representation. The node subscribes to the robot’s odometry, the global occupancy map, and high-level state machine commands, and publishes exploration goal points together with visualization markers. The exploration process can be activated or paused according to external commands, enabling integration within the overall autonomous system architecture.

Frontier regions are defined as boundaries between explored free space and unknown areas in the OctoMap. These frontier points represent potential exploration targets where new environment information can be obtained. Candidate frontier points are extracted within a bounded region around the UAV.

Once a frontier target is selected, a path toward the goal is generated using classical motion planning algorithms. In our implementation, A* is used to compute an efficient path on the projected map, while RRT is applied to generate feasible trajectories in complex environments where obstacles may block direct paths. The resulting path is then forwarded to the trajectory generation module, which produces smooth flight trajectories for the controller to follow.

4.3. State machine

The state machine node is mainly accomplished by Zhuoya Zhang.

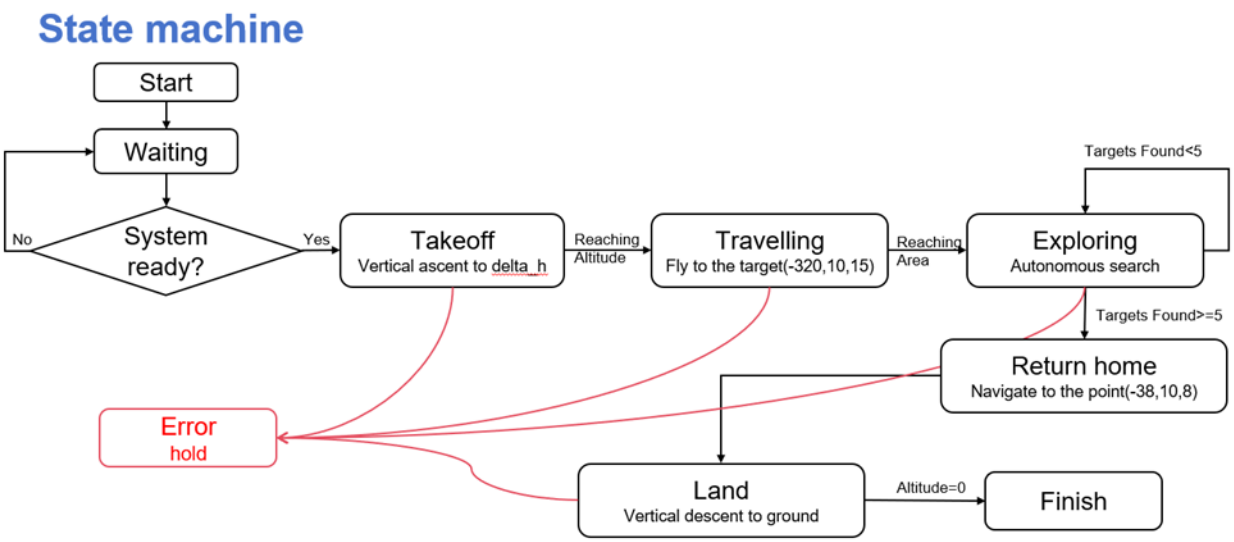


Figure 2. State machine logic

The system's decision-making is governed by a Finite State Machine that orchestrates the mission from deployment to recovery. The process begins with a WAITING state, transitioning to TAKEOFF and TRAVELLING once system health is confirmed. The core logic resides in the EXPLORING phase, where the UAV performs autonomous searching. A crucial logic branch is implemented here: the system continuously monitors the target counter, and only upon identifying 5 unique targets does it trigger the RETURN_HOME and LAND sequence. This ensures the mission objective is fully met before the UAV initiates its automated return-to-base protocol. For development and safety, the system incorporates a global ERROR state that can be triggered by any subsystem failure, ensuring the UAV enters a fail-safe hover mode.

4.4. Controller

The Controller Node("Controller_pkg") is applied from the basic version from Prof. Markus Ryll and Lukas Pries, further extended by Junda Zhu. The controller receives a desired trajectory and the current state estimate, computes the required total thrust and body moments using the control law, and finally converts them into individual rotor angular velocities. The control calculation strategies are based on "*Geometric tracking control of a quadrotor UAV on SE(3)*". There are four parameters included in the calculations, which are position gain(kx), velocity gain(kv), rotation gain(kr) and angular velocity gain(komega). These parameters can be adjusted in the file "controller_params.yaml". In addition to publishing motor commands, the controller also publishes a node health message. This signal is used by the state machine to monitor the operational status of the controller.

5. Conclusion

This project demonstrates a fully integrated autonomous UAV system capable of navigating and searching within a cave environment. By combining a vision-based perception pipeline with a robust Finite State Machine, the drone successfully identifies mission targets and executes an automated return-to-base sequence. This modular architecture highlights the effectiveness of decoupling high-level mission logic from low-level trajectory generation and control for reliable autonomous operations.

However, the system does not achieve perfect stability in every run. Due to the inherent challenges of underground environments, such as limited visibility and performance may vary under different conditions. Occasional exploration inefficiencies or transient localization inconsistencies were observed, indicating that further refinement in perception robustness and parameter tuning could improve overall reliability.

Bibliography

[1] “gmapping - ROS Wiki.” http://wiki.ros.org/gmapping (accessed Aug. 02, 2022).

[2] “octomap_mapping - ROS Wiki.” http://wiki.ros.org/octomap_mapping (accessed Aug. 02, 2022).

[3] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An efficient probabilistic 3D mapping framework based on octrees," *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.

[4] R. B. Rusu and S. Cousins, "3D is here: Point Cloud Library (PCL)," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Shanghai, China, 2011, pp. 1–4.

[5] S. S. Belavadi, S. Li, and K. Gupta, "Frontier exploration technique for 3D autonomous SLAM using K-means based divisive clustering," in *Proc. Asia Modelling Symp. (AMS)*, Kota Kinabalu, Malaysia, 2017, pp. 10–15.

[6] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proc. IEEE Int. Symp. Comput. Intell. Robot. Autom. (CIRA)*, Monterey, CA, USA, 1997, pp. 146–151.

[7] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Shanghai, China, 2011, pp. 2520–2525.